VIETNAM NATIONAL UNIVERSITY

HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

Digital Systems

Lab 3

Instructor: Đoàn Ngọc Cẩm

Student	Student ID
Hồ Chí Dũng	2351016
Nguyễn Viết Thi	2451165
Trần Duy Anh	2351007

Ho Chi Minh City – February 2026

OBJECTIVES

- Understand the representation of 8-bit floating-point numbers.
- Become familiar with describing a floating-point adder and subtractor.
- Design an 8-bit floating-point adder/subtractor.
- Download the circuit into the FPGA chip and test its functionality.

PREPARATION FOR LAB 3

- Finish Pre Lab 3 at home.
- Students must simulate all the exercises in Pre Lab 3 at home. All results (codes, waveform, RTL viewer, …) must be captured and submitted to instructors prior to the lab session.

  If not, students will not participate in the lab and be considered absent this session.

REFERENCE

EXPERIMENT

Objective: Design and implement a floating-point adder/subtractor in SystemVerilog, then download the circuit to the FPGA chip.

Requirements:

In lab 3, you are required to design a floating-point adder/subtractor should perform correctly in normal cases. Besides, several pins need to indicate some extreme cases:

- Zero detection: when the result is zero, zero detection pin will be 1.
- Overflow detection: when the result is out of range, the overflow occurs. The overflow detection will be 1 to indicate that an overflow has occurred, and the result is not reliable.

Inputs	Operand 1	A[7:0]	8-bit normalized input
	Operand 2	B[7:0]	8-bit normalized input
	Selection	S	1-bit input: addition/subtraction selection
Outputs	Operation result	Result[7:0]	8-bit normalized output
	Zero detection	Z	1-bit output
	Overflow detection	OV	1-bit output

Table 1: IO definition

Instruction:

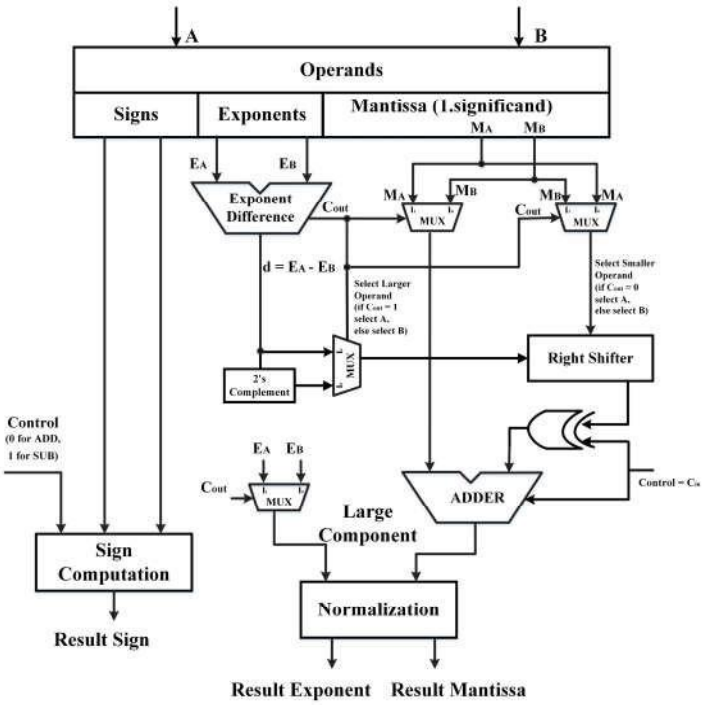
Input A and B have 8 bits, in which the sign is represented by bit [7], the exponent value is represented by bit [6:4], and the remaining is for Fraction value. Output Result also needs to be normalized as input signal.

The design has some sub-modules that perform floating point calculations:

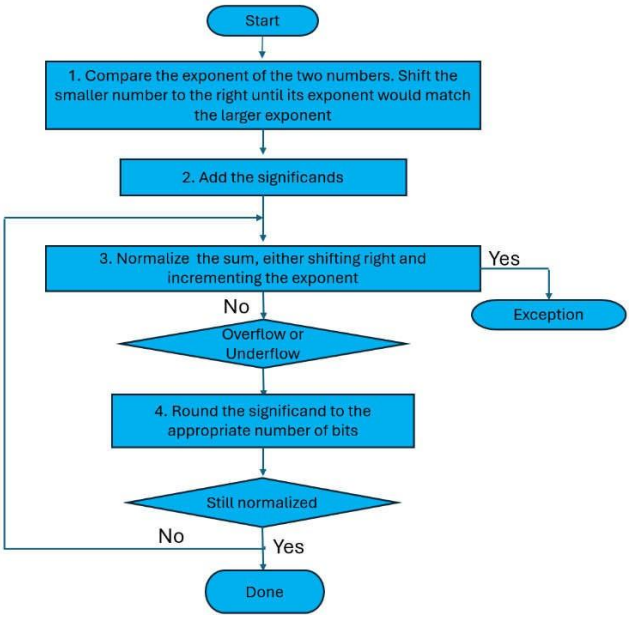
- Identify which number is larger, which number is smaller.
- Identify the amount to shift the operand which has smaller exponent.

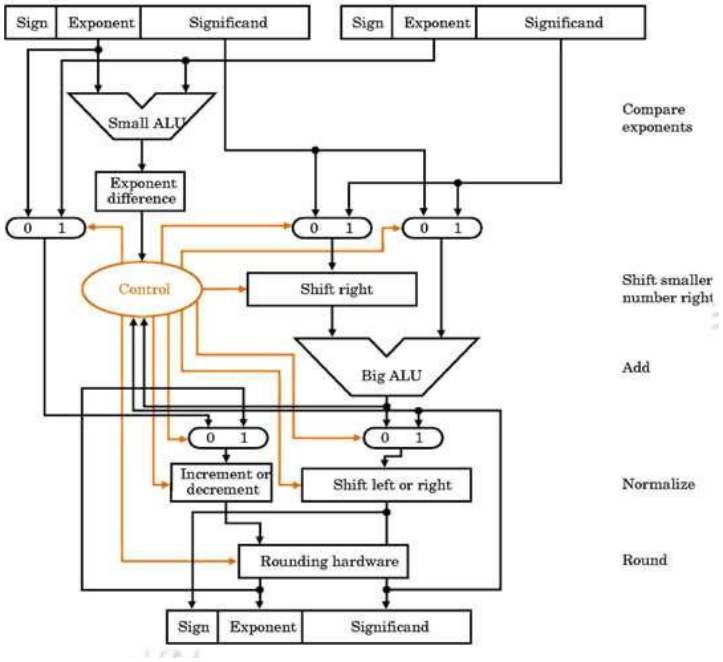
- Right shift fraction value of the smaller operand to align decimal points. - Calculate the two's complement of the shifted fraction, only needed in the case of subtraction or equivalent case (i.e. adding a negative number to a positive number).
- Add the two fractions together.
- Normalize the fraction and exponent value so it's back in floating point representation.
- Determine sign of the final value.
- Detect zero: the result is zero if the signs of A and B are different and there is no difference in the fraction and exponent.

The following image demonstrates a Floating-Point Adder/Subtractor structure:



Student may integrate a controller with the above combinational circuit, a “Done” signal will be asserted, indicating the FPU has finished computation.





1. Create a new Quartus project for your circuit.
2. Use switches as inputs, LEDRs as outputs.
3. Compile your project. Download the circuit into the FPGA chip and test its functionality.

Check: Your report must show two results:

- The waveform to prove the circuit works correctly.
- The result of RTL viewer.

Code

```
module l3 (
    input   logic [1:0] KEY,    // KEY[1]: Clock, KEY[0]: Reset
    input   logic [9:0] SW,     // SW[9]: Sel, SW[8]: Load, SW[7:0]: Data
    output  logic [9:0] LEDR,   // LEDR[9]: OV, LEDR[8]: Zero, LEDR[7:0]: Result
    output  logic [6:0] HEX0    // HEX0: Hiển thị chữ 'd'
);
    logic [7:0] reg_A, reg_B, Sum_float;
    logic overflow, underflow, Zero;
    logic [1:0] press_count;
    logic done_reg;

    // Zero Detection: Kiểm tra 8 bit kết quả (chấp nhận cả +0 và -0)
    assign Zero = (LEDR[7:0] == 8'b00000000 || LEDR[7:0] == 8'b10000000);

    // Done Controller: Nhấn KEY[1] 3 lần để hiện chữ 'd'
    always_ff @(posedge KEY[1] or negedge KEY[0]) begin
        if (!KEY[0]) begin
            press_count <= 2'd0;
            done_reg    <= 1'b0;
        end else begin
            if (press_count < 2'd3) press_count <= press_count + 1'b1;
            if (press_count == 2'd2) done_reg <= 1'b1;
        end
    end
    assign HEX0 = done_reg ? 7'b0100001 : 7'b1111111;

    // Hệ thống thanh ghi nạp dữ liệu
    register_8_bit rA(.clk(KEY[1]), .rst(KEY[0]), .D(SW[8] ? SW[7:0] : reg_A),
.Q(reg_A));
    register_8_bit rB(.clk(KEY[1]), .rst(KEY[0]), .D(!SW[8] ? SW[7:0] : reg_B),
.Q(reg_B));

    // Thanh ghi đầu ra (Chốt kết quả sau khi tính toán)
    register_8_bit rS(.clk(KEY[1]), .rst(KEY[0]), .D(Sum_float), .Q(LEDR[7:0]));
    register_1_bit rO(.clk(KEY[1]), .rst(KEY[0]), .D(overflow),  .Q(LEDR[9]));
    register_1_bit rZ(.clk(KEY[1]), .rst(KEY[0]), .D(Zero),      .Q(LEDR[8]));

    prelab3_top_add fpu_core(
        .Sel(SW[9]), .Float_A(reg_A), .Float_B(reg_B),
        .Sum_float(Sum_float), .overflow(overflow), .underflow(underflow)
    );
endmodule

// ============================================================================
// 2. FPU CORE: Xử lý cộng/trừ số thực 8-bit
```

```
// ============================================================================
module prelab3_top_add(
    input   logic        Sel,
    input   logic [7:0] Float_A, Float_B,
    output  logic [7:0] Sum_float,
    output  logic        overflow, underflow
);
    logic sign_A, sign_B, Result_sign, true_A_bigger;
    logic [2:0] Exp_A, Exp_B, Final_exp, Result_exp, eff_A, eff_B;
    logic [3:0] Fract_A, Fract_B, Result_fract;
    logic [9:0] mant_A, mant_B, mant_small_shifted, temp_sum;
    logic [2:0] Exp_diff;

    assign {sign_A, Exp_A, Fract_A} = Float_A;
    assign {sign_B, Exp_B, Fract_B} = Float_B;

    // FIX: Xử lý số mũ hiệu dụng cho Subnormal (Exp=0)
    assign eff_A = (Exp_A == 3'b0) ? 3'b001 : Exp_A;
    assign eff_B = (Exp_B == 3'b0) ? 3'b001 : Exp_B;

    // So sánh độ lớn tuyệt đối để tìm số làm mốc
    assign true_A_bigger = (Float_A[6:0] >= Float_B[6:0]);
    assign Exp_diff  = true_A_bigger ? (eff_A - eff_B) : (eff_B - eff_A);
    assign Final_exp = true_A_bigger ? Exp_A : Exp_B;

    // Hidden bit: Normal = 1, Subnormal = 0
    logic hA, hB;
    assign hA = (Exp_A == 3'b0) ? 1'b0 : 1'b1;
    assign hB = (Exp_B == 3'b0) ? 1'b0 : 1'b1;

    // Mở rộng Mantissa ra 10 bit để giữ độ chính xác [001.Fract_00]
    assign mant_A = {2'b0, (true_A_bigger ? hA : hB), (true_A_bigger ? Fract_A :
Fract_B), 3'b0};
    assign mant_B = {2'b0, (true_A_bigger ? hB : hA), (true_A_bigger ? Fract_B :
Fract_A), 3'b0};
    assign mant_small_shifted = mant_B >> Exp_diff;

    // Phép toán thực tế (Cộng hay Trừ mantissa)
    logic eff_sub;
    assign eff_sub = Sel ^ sign_A ^ sign_B;
    assign temp_sum = eff_sub ? (mant_A - mant_small_shifted) : (mant_A +
mant_small_shifted);

    // Chuẩn hóa kết quả
    prelab3ex4_structural normalizer (
        .Temp_10bit(temp_sum), .Final_exp(Final_exp),
        .Result_fract(Result_fract), .Result_exp(Result_exp),
        .overflow(overflow), .underflow(underflow)
    );
```

```
    // Xác định dấu kết quả
    always_comb begin
        if (Sel == 0) Result_sign = true_A_bigger ? sign_A : sign_B;
        else          Result_sign = true_A_bigger ? sign_A : !sign_B;
    end

    assign Sum_float = (Result_exp == 0 && Result_fract == 0) ? 8'b0 :
{Result_sign, Result_exp, Result_fract};
endmodule

// ============================================================================
// 3. NORMALIZATION: Tìm bit 1 đầu tiên và dịch chuyển
// ============================================================================
module prelab3ex4_structural (
    input  logic [9:0] Temp_10bit,
    input  logic [2:0] Final_exp,
    output logic [3:0] Result_fract,
    output logic [2:0] Result_exp,
    output logic underflow, overflow
);
    logic [9:0] norm_val;
    logic signed [4:0] shift_amt;

    always_comb begin
        if (Temp_10bit[8]) begin      // Có Carry (vídụ 1.x + 1.x)
            norm_val = Temp_10bit >> 1;
            shift_amt = 1;
        end else if (Temp_10bit[7]) begin // Đúng chuẩn (1.x)
            norm_val = Temp_10bit;
            shift_amt = 0;
        end else if (Temp_10bit[6]) begin // Dịch trái 1
            norm_val = Temp_10bit << 1;
            shift_amt = -1;
        end else if (Temp_10bit[5]) begin // Dịch trái 2
            norm_val = Temp_10bit << 2;
            shift_amt = -2;
        end else if (Temp_10bit[4]) begin // Dịch trái 3
            norm_val = Temp_10bit << 3;
            shift_amt = -3;
        end else begin
            norm_val = 0; shift_amt = 0;
        end
    end

    logic signed [4:0] exp_calc;
    assign exp_calc = {2'b0, Final_exp} + shift_amt;

    always_comb begin
```

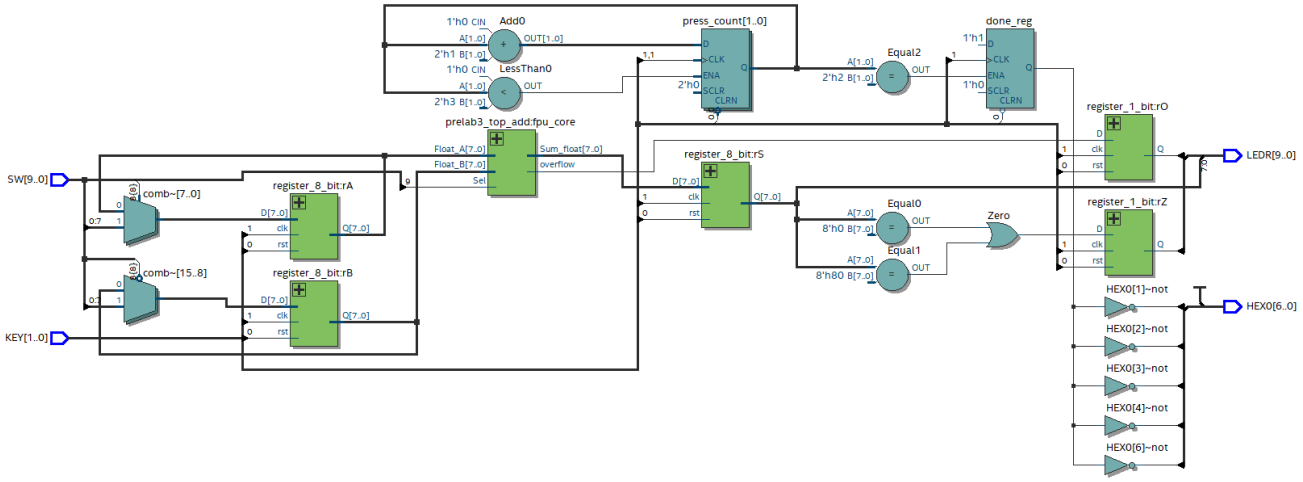
```
        if (Temp_10bit[8:3] == 6'b0) begin // Kết quả là Zero
            {overflow, underflow, Result_exp, Result_fract} = 9'b0;
        end else if (exp_calc > 5'sd6) begin // Infinity
            {overflow, underflow, Result_exp, Result_fract} = {1'b1, 1'b0,
3'b111, 4'b0000};
        end else if (exp_calc < 5'sd0) begin // Underflow (Số quá nhỏ)
            {overflow, underflow, Result_exp, Result_fract} = {1'b0, 1'b1,
3'b000, 4'b0000};
        end else begin
            {overflow, underflow, Result_exp, Result_fract} = {1'b0, 1'b0,
exp_calc[2:0], norm_val[6:3]};
        end
    end
endmodule

// Helper Modules
module register_8_bit(input logic clk, rst, input logic [7:0] D, output logic
[7:0] Q);
    always_ff @(posedge clk or negedge rst) if(!rst) Q <= 8'b0; else Q <= D;
endmodule

module register_1_bit(input logic clk, rst, input logic D, output logic Q);
    always_ff @(posedge clk or negedge rst) if(!rst) Q <= 1'b0; else Q <= D;

   endmodule
```

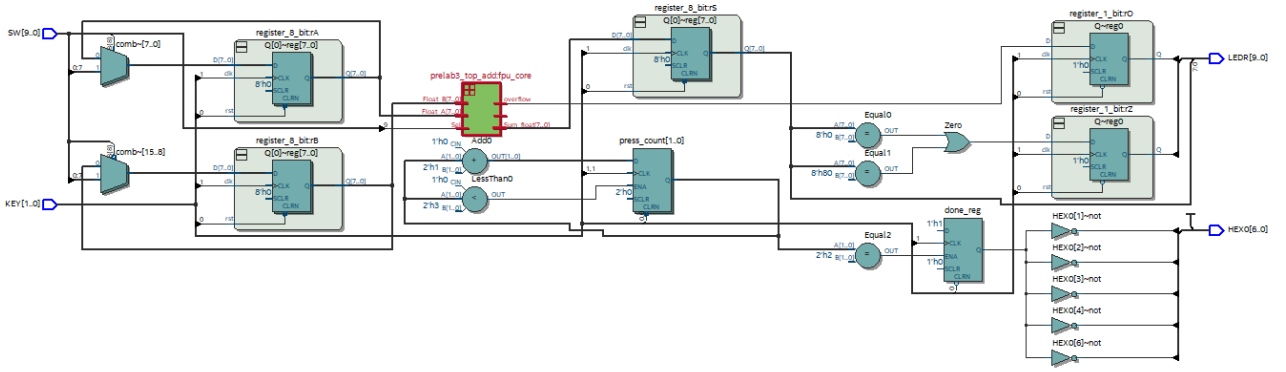
Testbench

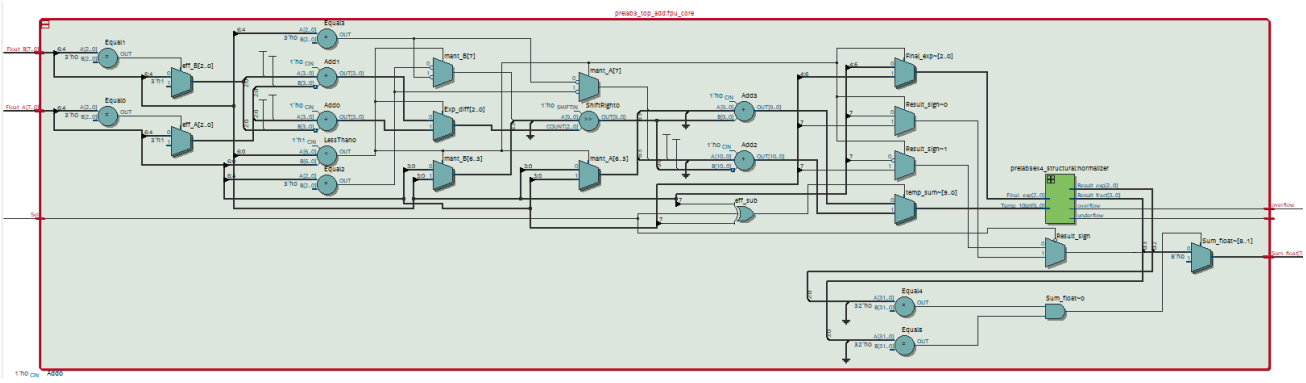
```
module l3_tb;
    logic [1:0] KEY;
    logic [9:0] SW;
    logic [9:0] LEDR;
    logic [6:0] HEX0;

    l3 dut (.*);

    // Task mô phỏng nhấn nút KEY[1]
    task press;
    begin
        KEY[1] = 0; #10;
        KEY[1] = 1; #10;
    end
    endtask

    // Task Reset
    task reset_dut;
    begin
        KEY[0] = 0; KEY[1] = 1; SW = 0; #20;
        KEY[0] = 1; #10;
```

```
    end
    endtask

    task run_test(input [7:0] a, input [7:0] b, input sel, input string name);
    begin
        $display("\nTEST: %s", name);
        reset_dut();

        // Nhịp 1: Nạp số A
        SW[9]=sel; SW[8]=1; SW[7:0]=a; press();

        // Nhịp 2: Nạp số B
        SW[8]=0; SW[7:0]=b;            press();

        // Nhịp 3: Chốt kết quả FPU ra thanh ghi LEDR
        press();

        #5;
        $display("A: %b | B: %b | Op: %s", a, b, sel ? "SUB" : "ADD");
        $display("RESULT:  %b | Zero: %b | OV: %b", LEDR[7:0], LEDR[8], LEDR[9]);
        #10;
    end
    endtask

    initial begin
        // 1. Cơ bản: 1.5 + 0.75 = 2.25 (0_100_0010)
        run_test(8'b0_011_1000, 8'b0_010_1000, 0, "1.5 + 0.75");

        // 2. Subnormal: 0.25 + 0.015625 = 0.265625 (0_001_0001)
        run_test(8'b0_001_0000, 8'b0_000_0001, 0, "Subnormal Case");

        // 3. Infinity: Tràn số mũ
        run_test(8'b0_110_1111, 8'b0_010_0000, 0, "Infinity Case");

        // 4. Zero: 1.25 - 1.25 = 0
        run_test(8'b0_011_0100, 8'b0_011_0100, 1, "Zero Case");

       // 5. Subtraction: 5.5 - 1.25 (Kỳ vọng: 4.25 -> 0_101_0001)
        run_test(8'b0_101_0110, 8'b0_011_0100, 1, "Subtraction: 5.5 - 1.25");

        $display("\n--- HOÀN THÀNH KIỂM TRA ---");
        $stop;
    End
   endmodule
```

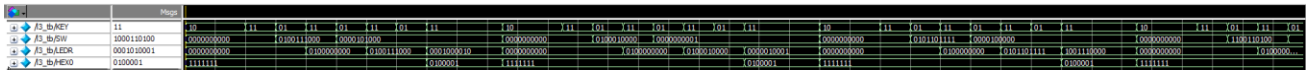
RTL


SW[9..0]
KEY[1..0]
comb~[7..0]
comb~[15..8]
register_8_bit:rA
register_8_bit:rB
prelab3_top_add:fpu_core
Float_A[7..0]
Float_B[7..0]
Sel
Sum_float[7..0]
overflow
register_8_bit:rS
D[7..0]
Q[7..0]
clk
rst
Add0
LessThan0
press_count[1..0]
Equal2
done_reg
register_1_bit:rO
register_1_bit:rZ
Equal0
Equal1
Zero
LEDR[9..0]
HEX0[1]~not
HEX0[2]~not
HEX0[3]~not
HEX0[4]~not
HEX0[6]~not
HEX0[6..0]


SW[9..0]
KEY[1..0]
comb~[7..0]
comb~[15..8]
register_8_bit:rA
register_8_bit:rB
prelab3_top_add:fpu_core
register_8_bit:rS
press_count[1..0]
Add0
LessThan0
Equal0
Equal1
Equal2
Zero
done_reg
register_1_bit:rO
register_1_bit:rZ
LEDR[9..0]
HEX0[1]~not
HEX0[2]~not
HEX0[3]~not
HEX0[4]~not
HEX0[6]~not
HEX0[6..0]


prelab3_top_add:fpu_core
Equal1
Equal3
Equal0
Equal2
Add0
Add1
Add2
Add3
LessThan0
eff_B[2..0]
eff_A[2..0]
mant_B[7]
mant_B[6..3]
mant_A[7]
mant_A[6..3]
Exp_diff[2..0]
ShiftRight0
eff_sub
final_exp~[2..0]
Result_sign~0
Result_sign~1
temp_sum~[9..0]
prelab4_structural:normalizer
Result_sign
Sum_float~[8..1]
Sum_float~0
Equal4
Equal5

Simulation



Kit

1.Addtion: 1.5 + 0.75

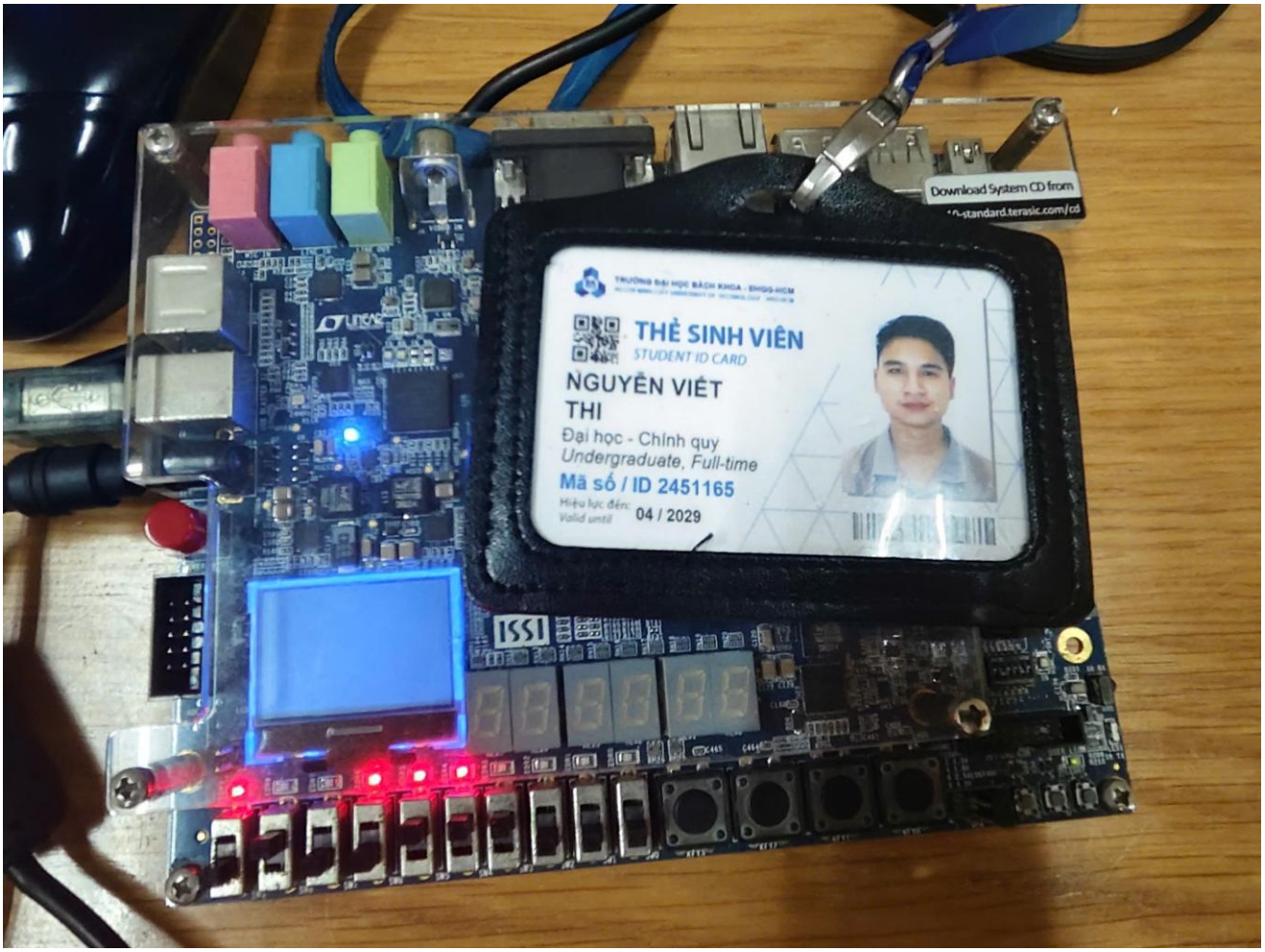


Figure 1. Input Floating point A = 00111000

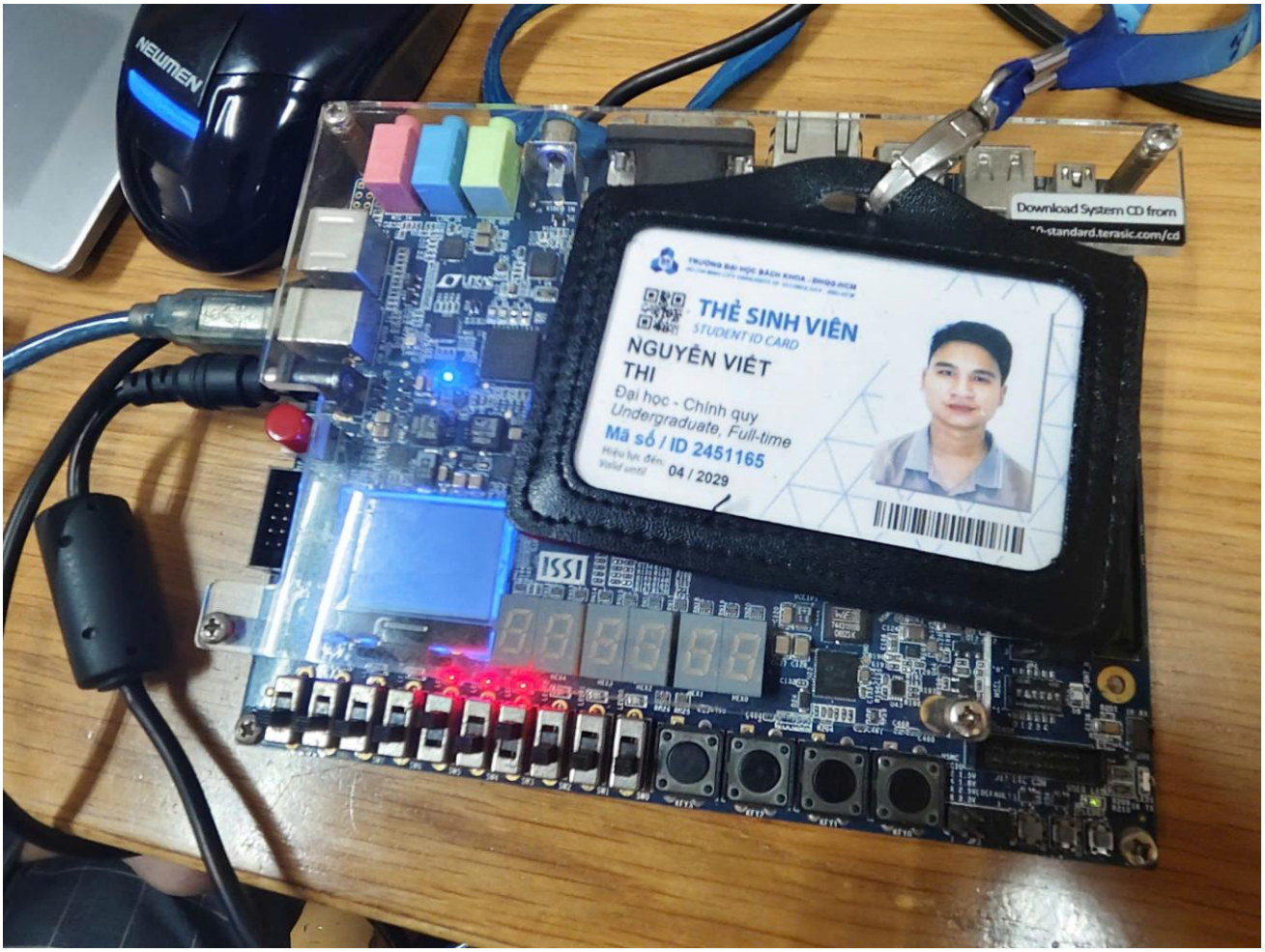


Figure . Input Floating point B = 0 010 1000

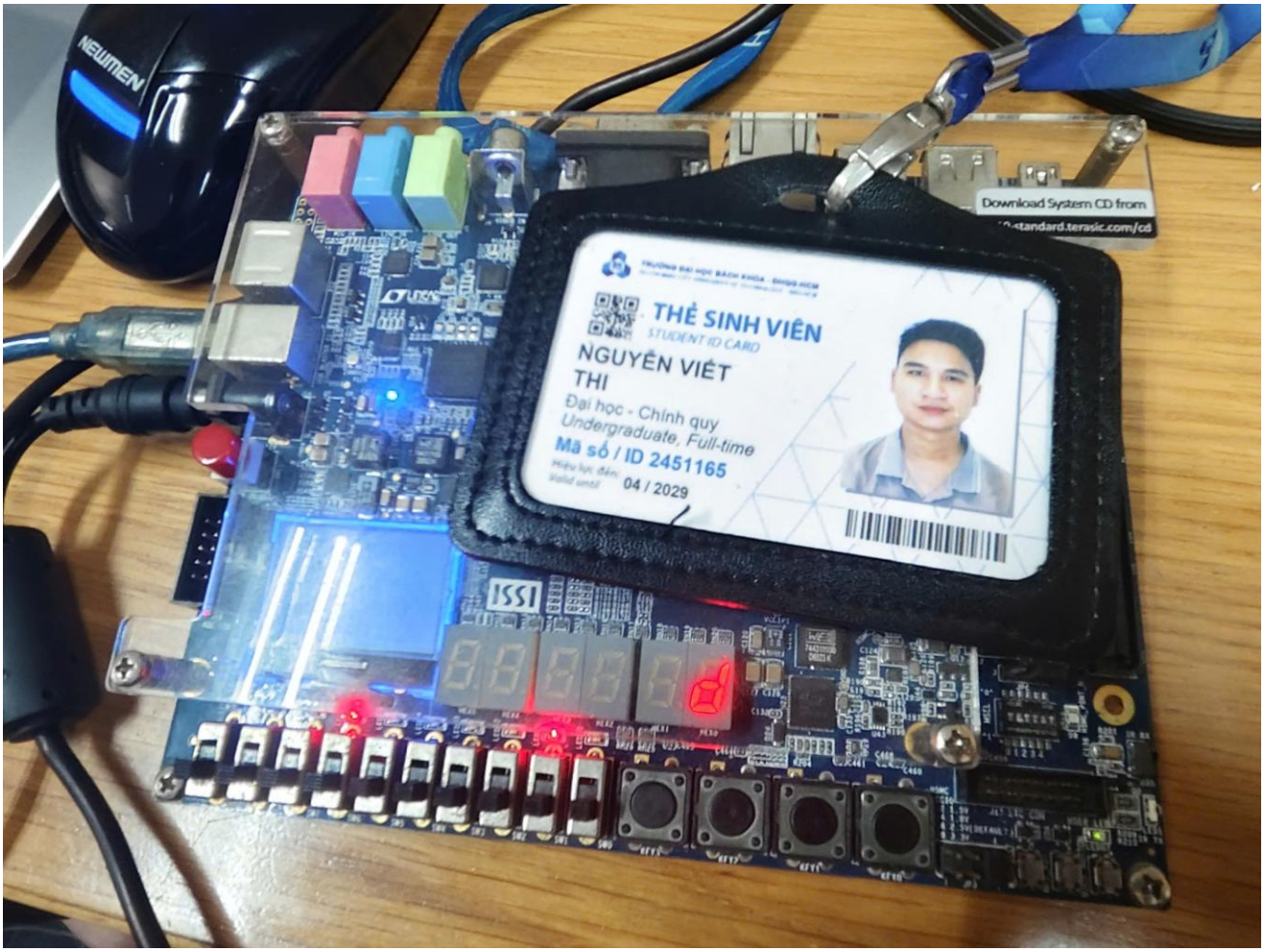


Figure .Sel = 0 => Output equal A + B = 01000010

2.Zero detection: 0.5 – 0.5

Figure . Input Floating point A = 0 000 0001

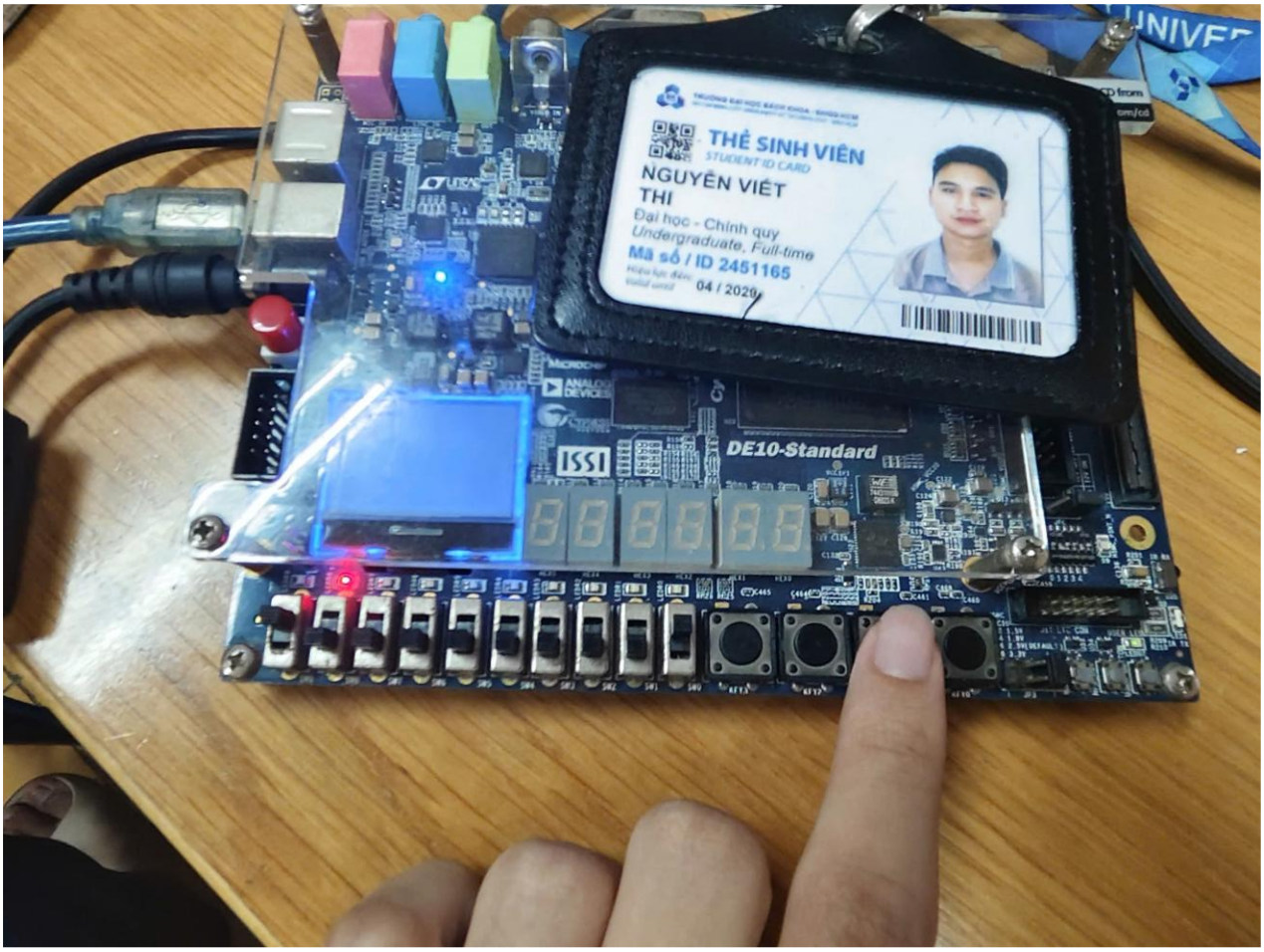


Figure . Input Floating point B = 0 000 0001

Figure. Zero detection => LEDR[8] turns on (Because A-B=0, Sel = 0)

3. Subtraction: 5.5 - 1.25

Figure . Input Floating point A = 0 101 0110

Figure . Input Floating point B = 0 011 0100

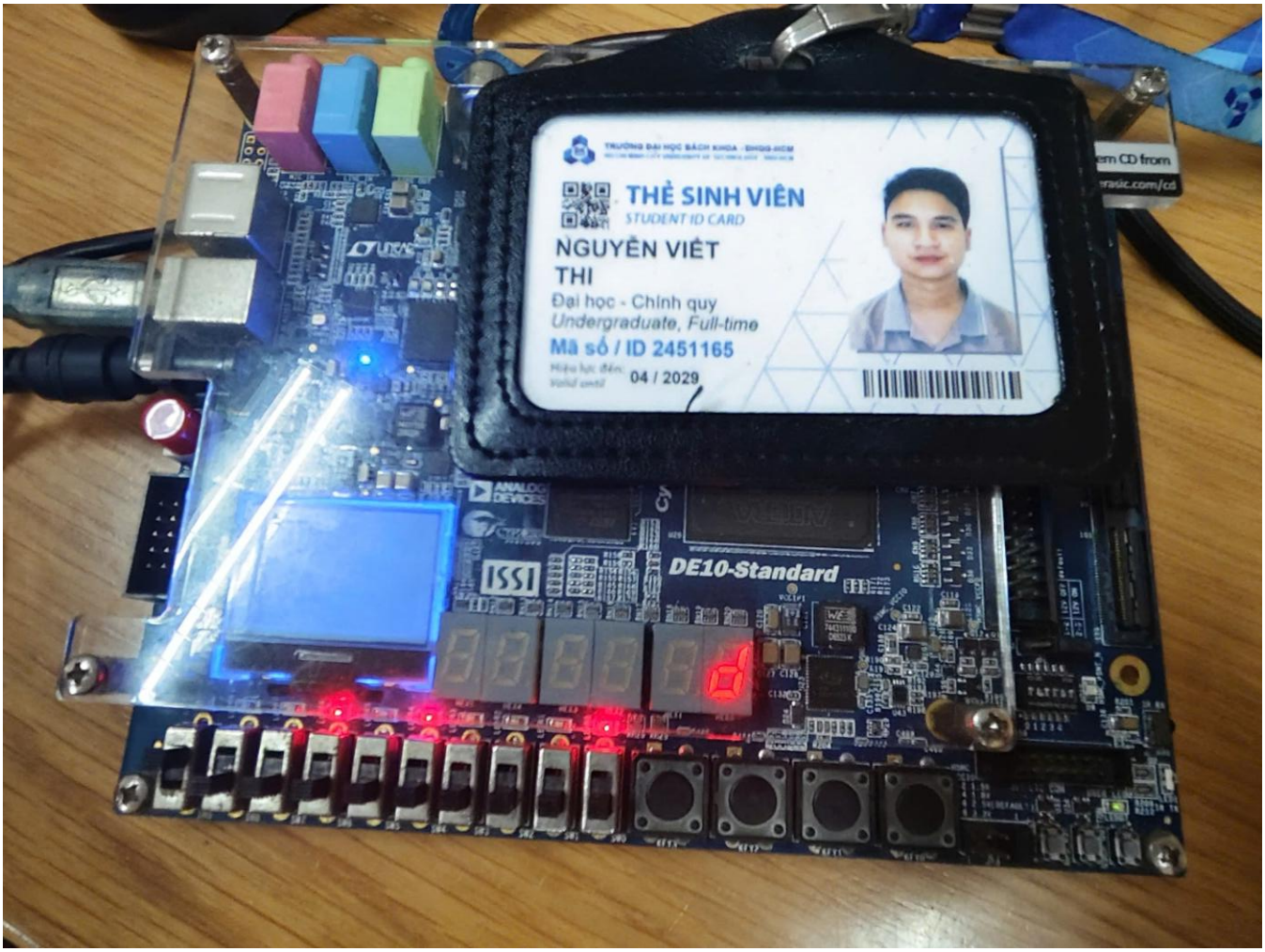


Figure .Sel = 1 => Output equal A - B = 0 101 0001

4. Addition with a subnormal number: 0.25 + 0.015625

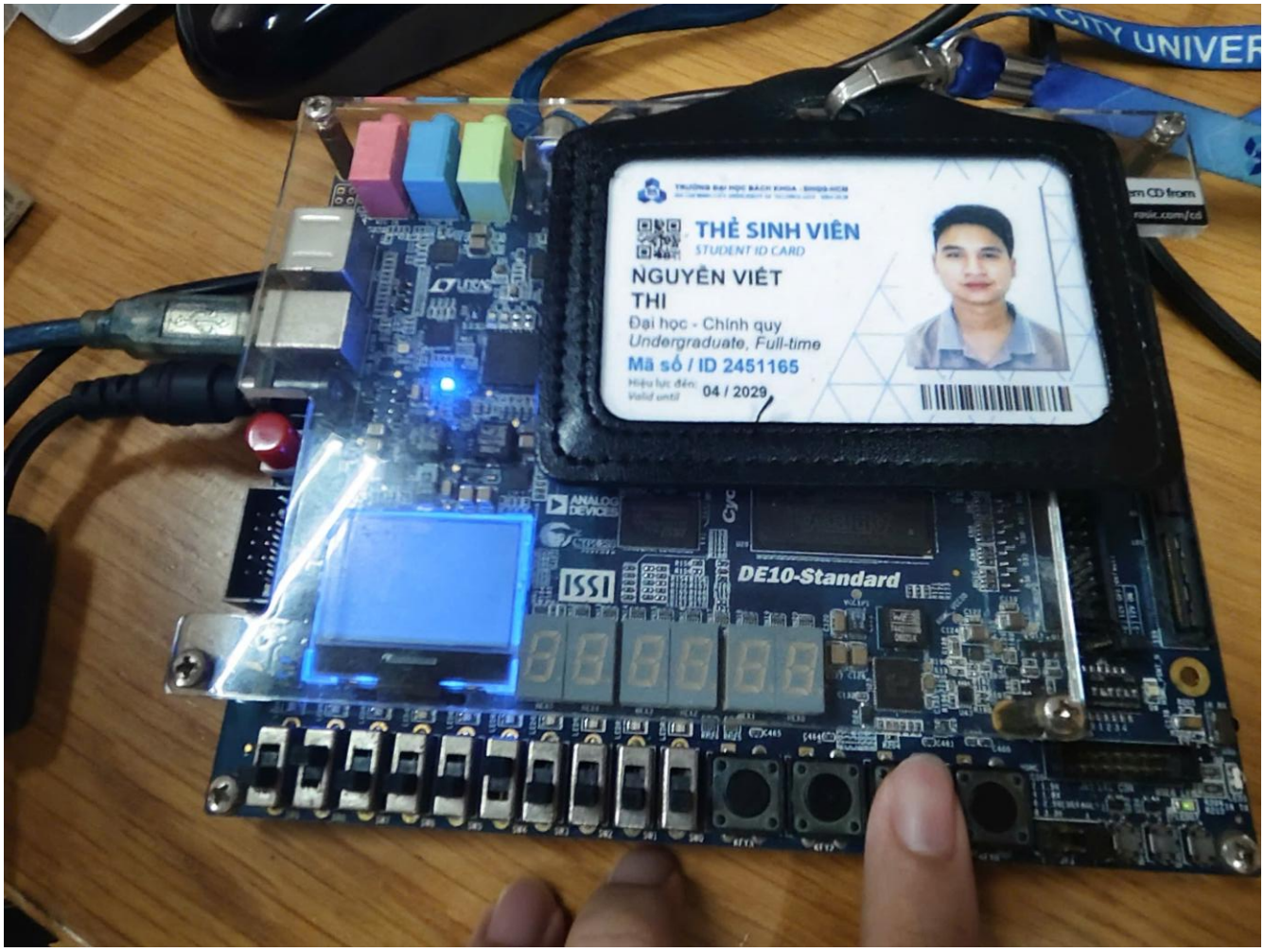


Figure . Input Floating point A = 0 001 0000

Figure . Input Floating point B = 0 000 0001

Figure .Sel = 0 => Output equal A + B = 0 001 0001

5. Addtion with inifinity result: 15.5 + 0.5

Figure . Input Floating point A = 0 110 1111

Figure . Input Floating point B = 0 010 0000

Figure .Sel = 0 => Output equal A + B = 0 111 0000